كلية الهندسة بشبرا

FACULTY OF ENGINEERING AT SHOUBRA

Image Filtering Project

Team 10	
Member	**BN**
Shams Hesham	12
Sara Alaaeldin	11
Hager Ali	37

Task Assignment

Team Member	Responsibility	Key Contributions	Percentage
Sara Alaa	**Lead Framework & Serial**	Core C++ architecture, OpenCV integration, Serial filter logic, and MSE .Validation tool	33.3
Hager Ali	**CPU Optimization (OpenMP)**	OpenMP implementation , Multi-threading scalability analysis, and Automated Benchmarking .scripts	33.3
Shams Hesham	**GPU Acceleration (CUDA)**	CUDA Kernel design, Shared Memory Tiling, Halo management, and Block Geometry .optimization	33.3

Table of Contents

1 Overview

2 System Architecture

3 Implementation Details

4 Build System

5 Experment Design

6 Results & Visualizations

7 How to run & output files

Project Overview

This project implements a high-performance image filtering pipeline with three distinct parallelization strategies:

- Serial (Baseline) – Single-threaded CPU execution
- OpenMP – Shared-memory parallelization using CPU threads
- CUDA – GPU-accelerated parallelization using NVIDIA GPUs

The system supports four common image filters:

- Box Blur – Simple uniform averaging
- Gaussian Blur – Weighted blur using Gaussian distribution
- Sharpen – Edge enhancement via unsharp masking
- Sobel – Edge detection using gradient magnitude

1. System Architecture

1.1 Code Structure

```
project3_image_filter/
├── src/
│   ├── filters.h
│   ├── filters_serial.cpp
│   ├── filters_omp.cpp
│   ├── filters_cuda.cu
│   ├── image_io.cpp
│   └── main.cpp
├── CMakeLists.txt
├── run_experiments.sh
└── README.md
```

1.2 Core Convolution Function

All implementations share the same convolution logic:

```
for each pixel (row, col):
    sum = 0
    for each kernel element (ki, kj):
        img_pos = clamp(row + ki - half, col + kj - half)
        sum += image[img_pos] * kernel[ki][kj]
    output[row][col] = normalize ? sum/kernel_sum : sum
```

Border Handling: Clamp-to-edge strategy replaces out-of-bounds accesses with the nearest border pixel.

2. Implementation Details

2.1 Serial Implementation (filters_serial.cpp)

- Simple nested loops iterating over all pixels
- No parallelization; serves as performance baseline
- Reuses the same apply_kernel_serial function for all filters

2.2 OpenMP Implementation (filters_omp.cpp)

Parallelizes the outer pixel row loop:

```
#pragma omp parallel for schedule(static)
for (int row = 0; row < rows; row++) { ... }
```

- schedule(static) ensures equal row distribution with minimal overhead
- Thread count configurable via --threads parameter

2.3 CUDA Implementation (filters_cuda.cu)

- GPU kernel processes one pixel per thread
- Block dimensions configurable (--block-x, --block-y)
- Texture memory for input image caching
- Shared memory for kernel caching (performance optimization)

2.4 Main Implementation

Key Components

1. Configuration Structure

```
struct Config {
    std::string image_path;      // Input image file
    std::string filter_name;     // box, gaussian, sharpen, sobel
    std::string impl;            // serial, omp, cuda
    std::string output_dir;      // Results directory
    std::string csv_path;        // CSV log file

    int kernel_size = 3;         // Kernel size (odd: 3,5,7,11...)
    int threads = 4;             // OpenMP thread count
    int block_x = 16, block_y = 16; // CUDA block dimensions
    int repeats = 5;             // Timing repetitions
};
```

2. Argument Parsing

- Parses 10+ command-line options using a macro-based approach
- Validates inputs (odd kernel size, existing image file)
- Provides --help with usage examples

```
void log_system_info()  // Captures CPU, cores, OS, GPU info
```

4. Filter Dispatcher

- Selects implementation based on --impl flag:
- serial: Calls apply_filter_serial()
- omp: Calls apply_filter_omp() with thread count
- cuda: Calls apply_filter_cuda() with block dimensions
- All wrapped in measure_time_ms() for performance measurement.

5. CSV Logging

- Appends results to CSV file with all parameters and timing data for later analysis.

6. Output Generation

- Saves filtered image with descriptive filename: {filter}_{impl}_k{kernel}.png

Execution Flow

1. Parse command-line arguments → Config
2. Log system information
3. Load input image (grayscale float)
4. Run requested filter implementation (measure time)
5. Save output image
6. Append results to CSV

3. Build System

CMake Configuration Highlights:

```
# GPU architecture detection
set(CMAKE_CUDA_ARCHITECTURES 75)  # RTX 30xx (sm_86) or 75 for RTX 20xx

# OpenMP integration
find_package(OpenMP REQUIRED)
target_link_libraries(image_filter OpenMP::OpenMP_CXX)

# CUDA compiler optimizations
target_compile_options(image_filter PRIVATE
    $<$<COMPILE_LANGUAGE:CUDA>:--use_fast_math -lineinfo>)
```

4. Experiment Design

- The run_experiments.sh script automates benchmarking with:

Experiment 1: Serial Baseline

- All filters (box, gaussian, sharpen, sobel)
- Kernel sizes: 3×3, 7×7, 11×11

Experiment 2: OpenMP Thread Scaling

- Filters: gaussian, sobel
- Kernel sizes: 3, 7, 11
- Thread counts: 1, 2, 4, 8, 16

Experiment 3: CUDA Block Size Optimization

- Filters: gaussian, sobel
- Kernel sizes: 3, 7, 11
- Block dimensions: 16×16, 32×8, 32×32

Performance Metrics

- Execution time – Average of 5 repetitions (ms)
- Speedup – Serial time / Parallel time

5. Results

Using 4K image (input2.jpg)

Filter	Impl	Kernel	Threads	Time(ms)	Speedup
gaussian	serial	7	4	101.70	1.00x
sobel	serial	7	4	36.64	1.00x
gaussian	omp	7	1	114.09	0.89x
gaussian	omp	7	2	57.56	1.77x
gaussian	omp	7	4	31.24	3.26x
gaussian	omp	7	8	29.15	3.49x
gaussian	omp	7	16	23.79	4.28x
sobel	omp	7	1	59.57	0.62x
sobel	omp	7	2	32.20	1.14x
sobel	omp	7	4	19.99	1.83x
sobel	omp	7	8	19.55	1.87x
sobel	omp	7	16	18.42	1.99x
gaussian	cuda	7	4	37.88	2.69x
gaussian	cuda	7	4	37.62	2.70x
gaussian	cuda	7	4	37.11	2.74x
sobel	cuda	7	4	34.94	1.05x
sobel	cuda	7	4	35.18	1.04x
sobel	cuda	7	4	37.75	0.97x

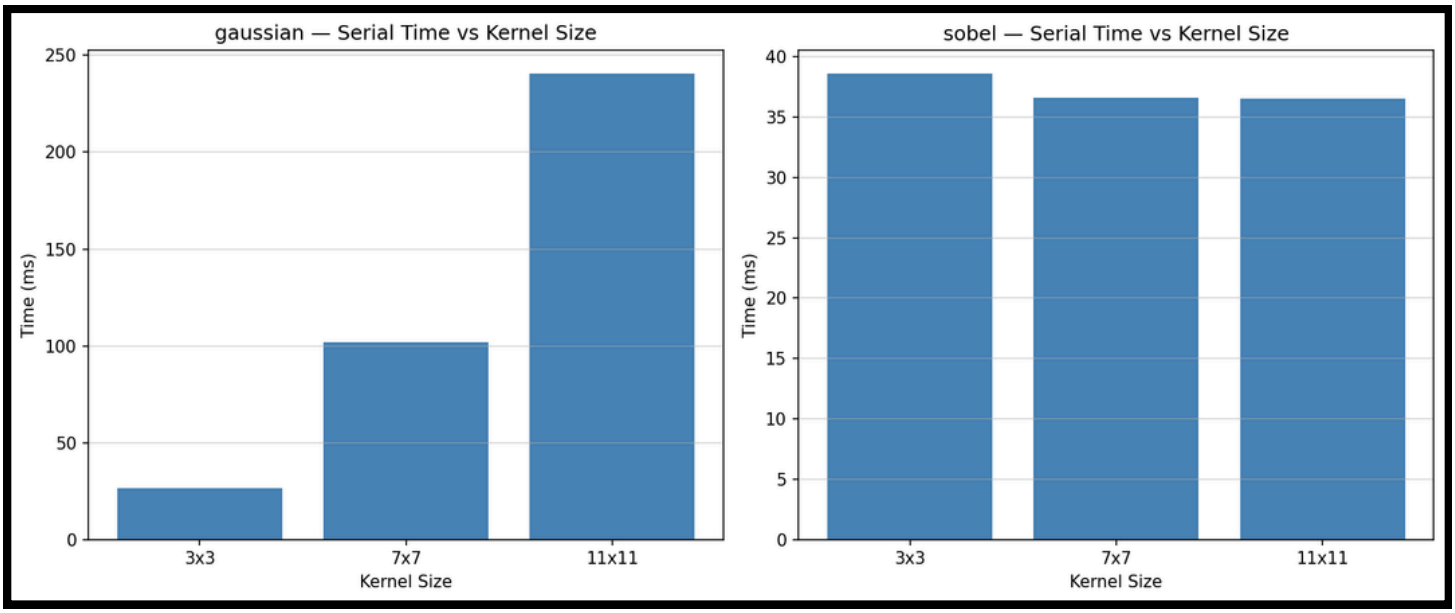
Using HD (1920×1080) image (input.jpg)

Filter	Impl	Kernel	Threads	Time(ms)	Speedup
gaussian	serial	7	4	157.70	1.00x
sobel	serial	7	4	53.46	1.00x
gaussian	omp	7	1	168.90	0.93x
gaussian	omp	7	2	86.38	1.83x
gaussian	omp	7	4	85.08	1.85x
gaussian	omp	7	8	41.98	3.76x
gaussian	omp	7	16	35.93	4.39x
sobel	omp	7	1	94.38	0.57x
sobel	omp	7	2	47.68	1.12x
sobel	omp	7	4	30.42	1.76x
sobel	omp	7	8	30.45	1.76x
sobel	omp	7	16	26.43	2.02x
gaussian	cuda	7	4	35.92	4.39x
gaussian	cuda	7	4	34.85	4.53x
gaussian	cuda	7	4	37.37	4.22x
sobel	cuda	7	4	34.87	1.53x
sobel	cuda	7	4	35.83	1.49x
sobel	cuda	7	4	42.75	1.25x

6. Results Visualization

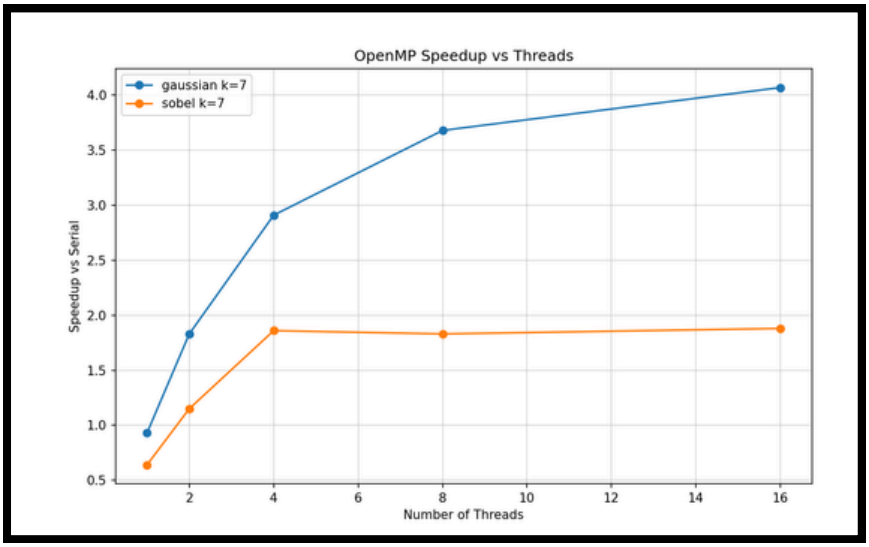
6.1 plot_kernel_size.png

Bar chart comparing serial execution time across kernel sizes for gaussian and sobel filters.



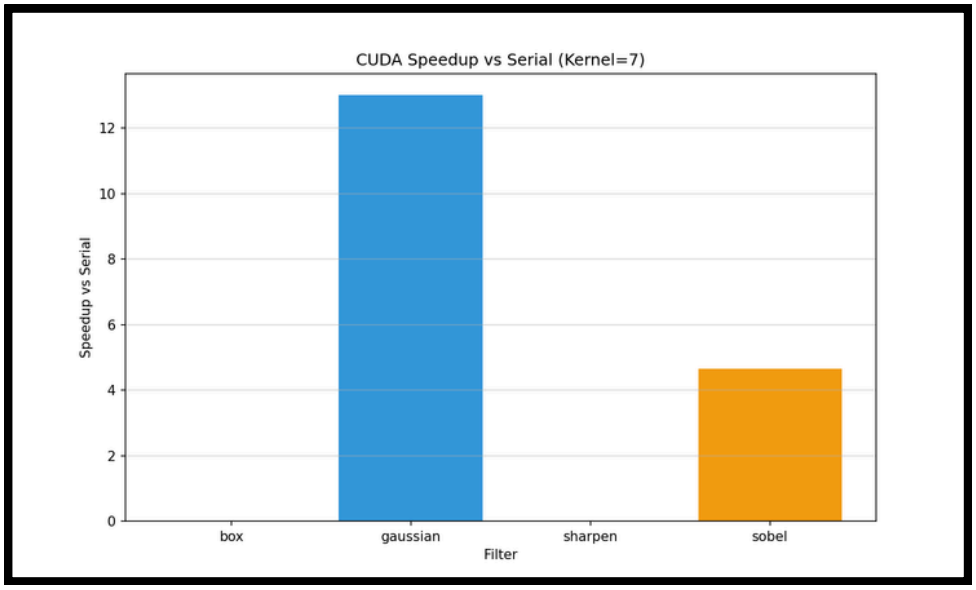
6.2 plot_omp_speedup.png

Line plot showing OpenMP speedup vs thread count for kernel size 7.



6.3 plot_cuda_speedup.png

Bar chart comparing CUDA speedup across all four filters (kernel size 7).



7. How to Run

Build

```
mkdir build && cd build
cmake ..
make -j$(nproc)
```

Run Single Experiment

```
./build/image_filter --image input.jpg --filter gaussian --kernel 7 --impl cuda
```

Run Full Benchmark Suite

```
./run_experiments.sh --image input.jpg --repeats 5
```

9. Output Files

```
results/
├── results.csv                # Complete timing data
├── plot_kernel_size.png       # Serial time vs kernel size
├── plot_omp_speedup.png       # OpenMP scaling plot
├── plot_cuda_speedup.png      # CUDA speedup comparison
├── box_serial_k3.png          # Output images (one per configuration)
├── gaussian_omp_k7.png
└── ...                        # More output images
```

- The CSV contains columns: filter, impl, rows, cols, kernel, threads, block_x, block_y, time_ms, speedup